# PCD - Homework 1

Comparative Analysis of Data Transfer Performance
Using TCP, UDP, and QUIC Protocols

Baroncea Andrei-Florin

# 1 Introduction

This report presents a comparative analysis of data transfer performance across three transport protocols: TCP, UDP, and QUIC. The experiments measure transmission time, throughput, and data loss when transferring 512 MB and 1 GB of data using five different block sizes (12 000, 24 000, 36 000, 48 000, and 60 000 bytes). Two transfer mechanisms are evaluated: *streaming* (continuous transfer) and *stop-and-wait* (per-message acknowledgment).

Data integrity is verified using SHA-256 hashing on both the client and the server side.

# 2 Program Usage

The application consists of four Python files:

- `common.py` - shared constants, utility functions, and certificate generation.
- `server.py` - asynchronous server supporting TCP, UDP, and QUIC.
- `client.py` - client supporting TCP, UDP, and QUIC.
- `run_tests.py` - automated benchmark runner.

## 2.1 Dependencies

Install the required packages:

```
pip install aioquic cryptography
```

## 2.2 Server

```
python server.py -protocol <tcp|udp|quic>
                  [-host HOST] [-port PORT]
```

| Option | Default | Description |
|---|---|---|
| `-protocol` | (required) | Transport protocol: `tcp`, `udp`, or `quic` |
| `-host` | `0.0.0.0` | Bind address |
| `-port` | `12345` | Listen port |

All three servers are implemented using `asyncio`, allowing concurrent client handling:

- **TCP**: `asyncio.start_server` with per-connection coroutines.
- **UDP**: `asyncio.DatagramProtocol` with per-address session queues.
- **QUIC**: `aioquic` `serve()` with per-stream processing coroutines.

At the end of each session the server prints: protocol used, total messages received, total bytes received, and the SHA-256 hash.

### 2.3 Client

```
python client.py -protocol <tcp|udp|quic>
                 -size SIZE_MB
                 -block-size BLOCK_SIZE
                 -mode <streaming|stop-and-wait>
                 [-host HOST] [-port PORT]
```

| Option | Default | Description |
|---|---|---|
| `-protocol` | (required) | Transport protocol |
| `-size` | (required) | Data size in megabytes |
| `-block-size` | (required) | Block size in bytes (1-65 535) |
| `-mode` | (required) | `streaming` or `stop-and-wait` |
| `-host` | `127.0.0.1` | Server address |
| `-port` | `12345` | Server port |

At the end of each transfer the client prints: protocol, mode, block size, total time, throughput, messages sent/received/lost, bytes sent/received/lost, data loss percentage, and SHA-256 hashes from both sides with a match indicator.

### 2.4 Automated Benchmarks

```
python run_tests.py
```

This script iterates over all combinations of protocols, modes, block sizes, and data sizes. Each test starts a fresh server on a unique port, runs the client, and logs results both to the console and to `results.txt`. The test timeout is set to 7 200 s (2 hours) per test.

### 2.5 Example

```
# Terminal 1: Start server
python server.py -protocol tcp -port 13000

# Terminal 2: Run client
python client.py -protocol tcp -port 13000 \
    -size 512 -block-size 60000 -mode streaming
```

## 3 Implementation Details

### 3.1 Data Generation

The client generates a reusable block of the requested size from a repeating 0-255 byte pattern. Both sides compute an incremental SHA-256 hash over the transferred payload. After the transfer completes, the server sends back its hash and the client compares them.

## 3.2 Transfer Mechanisms

### 3.2.1 Streaming

Data is sent continuously without waiting for per-message acknowledgments:

- **TCP**: the client calls `sendall()` in a loop, relying on TCP's built-in flow control.
- **UDP**: the client sends datagrams as fast as possible with no delivery guarantee.
- **QUIC**: the client pushes data into the QUIC stream buffer and calls `transmit()` to flush.

### 3.2.2 Stop-and-Wait

Each message requires an explicit acknowledgment before the next is sent:

- **TCP**: the client sends `[4-byte length][payload]`, then blocks until the server responds with a 1-byte ACK (`0x06`). A zero-length header signals end of transfer.
- **UDP**: the client sends `[MSG_DATA][4-byte seq][payload]` and waits for `[MSG_ACK][4-byte seq]`. If no ACK arrives within 2 s, the datagram is retransmitted (up to 10 retries).
- **QUIC**: identical framing to TCP (`[4-byte length][payload]` → wait for `0x06`).

## 3.3 QUIC Configuration

The QUIC implementation uses `aioquic`, a pure-Python QUIC library. The configuration includes:

- `idle_timeout` set to 1 200 s to prevent premature connection closure.
- Self-signed TLS certificates generated automatically via the `cryptography` library.

# 4 Experimental Results

All tests were performed on localhost to isolate protocol overhead from network latency. Block sizes tested: 12 000, 24 000, 36 000, 48 000, and 60 000 bytes. Data sizes: 512 MB and 1 024 MB (1 GB).

## 4.1 TCP - Streaming

Table 1: TCP streaming results

| Block (B) | 512 MB (s) | Speed (MB/s) | 1 GB (s) | Speed (MB/s) | Loss |
|---|---|---|---|---|---|
| 12 000 | 0.835 | 612.97 | 4.501 | 227.49 | 0% |
| 24 000 | 0.879 | 582.52 | 3.190 | 321.02 | 0% |
| 36 000 | 0.774 | 661.47 | 2.070 | 494.77 | 0% |
| 48 000 | 0.726 | 704.81 | 2.501 | 409.48 | 0% |
| 60 000 | 0.640 | 800.57 | 2.091 | 489.83 | 0% |

TCP achieves the highest and most consistent throughput. All hashes match and data loss is zero. Larger block sizes reduce the number of `sendall()` calls, improving speed from 613 to 801 MB/s at 512 MB.

## 4.2 UDP - Streaming

Table 2: UDP streaming results

| Block (B) | 512 MB (s) | MB/s | 1 GB (s) | MB/s | Loss 512 MB | Loss 1 GB |
|---|---|---|---|---|---|---|
| 12 000 | 3.480 | 147.12 | 4.895 | 209.19 | 0% | 37.88% |
| 24 000 | 1.357 | 377.42 | 3.869 | 264.68 | 0% | 16.88% |
| 36 000 | 0.856 | 598.31 | 2.741 | 373.59 | 0% | 10.12% |
| 48 000 | 0.686 | 746.19 | 12.461 | 82.18 | 0% | 0.97% |
| 60 000 | 0.641 | 798.50 | 2.137 | 479.20 | 0% | 0% |

At 512 MB, the entire transfer fits within OS socket buffers and there is zero loss. At 1 GB, smaller block sizes generate more datagrams, overwhelming the receiver's buffer. With 12 000-byte blocks, 37.88% of the data is lost. Larger blocks (60 000 B) reduce the total datagram count enough to avoid loss entirely, even at 1 GB.

## 4.3 UDP - Stop-and-Wait

Table 3: UDP stop-and-wait results

| Block (B) | 512 MB (s) | Speed (MB/s) | 1 GB (s) | Speed (MB/s) | Loss |
|---|---|---|---|---|---|
| 12 000 | 12.692 | 40.34 | 28.144 | 36.38 | 0% |
| 24 000 | 4.546 | 112.63 | 13.843 | 73.97 | 0% |
| 36 000 | 3.114 | 164.42 | 8.477 | 120.79 | 0% |
| 48 000 | 2.261 | 226.42 | 6.672 | 153.48 | 0% |
| 60 000 | 2.153 | 237.77 | 4.891 | 209.37 | 0% |

Stop-and-wait eliminates all data loss by requiring an ACK for every datagram. The trade-off is throughput: at 12 000-byte blocks, it is 3-6× slower than streaming because each message incurs a full round-trip delay. Increasing block size to 60 000 reduces the number of round trips from 44 740 to 8 948 (for 512 MB), improving speed nearly 6×.

### 4.4 QUIC - Streaming

Table 4: QUIC streaming results

| Block (B) | 512 MB (s) | Speed (MB/s) | 1 GB (s) | Speed (MB/s) | Loss |
|---|---|---|---|---|---|
| 12 000 | 438.57 | 1.17 | 877.87 | 1.17 | 0% |
| 24 000 | 438.36 | 1.17 | 876.93 | 1.17 | 0% |
| 36 000 | 438.39 | 1.17 | 877.55 | 1.17 | 0% |
| 48 000 | 438.54 | 1.17 | 877.92 | 1.17 | 0% |
| 60 000 | 438.49 | 1.17 | 877.59 | 1.17 | 0% |

The throughput is constant at **1.17 MB/s regardless of block size**. This demonstrates that the bottleneck is not the application-level block size but the `aioquic` library's internal processing: TLS 1.3 encryption/decryption and congestion control are implemented in pure Python, resulting in very high per-byte CPU overhead.

All successful transfers have matching hashes and zero data loss, confirming QUIC's reliability guarantees.

### 4.5 QUIC - Stop-and-Wait

Table 5: QUIC stop-and-wait results

| Block (B) | 512 MB (s) | Speed (KB/s) | 1 GB |
|---|---|---|---|
| 12 000 | 5 042.46 | 103.97 | TIMEOUT |
| 24 000 | 4 799.30 | 109.24 | TIMEOUT |
| 36 000 | 4 714.95 | 111.20 | TIMEOUT |

QUIC stop-and-wait is practically unusable for large transfers. Each round-trip passes through the full TLS encryption/decryption pipeline in pure Python, adding enormous per-message latency. At $\approx 100$ KB/s, a 1 GB transfer would require approximately 2.8 hours, exceeding the 2-hour test timeout. All completed 512 MB transfers have matching hashes.

## 5 Comparative Analysis

### 5.1 Throughput Comparison

Table 6: Throughput summary (MB/s) - best block size for each protocol

| Protocol + Mode | 512 MB (MB/s) | 1 GB (MB/s) |
|---|---|---|
| TCP streaming | 801 | 495 |
| UDP streaming | 799 | 479 |
| UDP stop-and-wait | 238 | 209 |
| QUIC streaming | 1.17 | 1.17 |
| QUIC stop-and-wait | 0.11 | — |

At their best, TCP and UDP streaming achieve comparable peak throughput ($\approx$ 800 MB/s at 60 KB blocks). UDP stop-and-wait reaches about 30% of streaming speed. QUIC is $\approx$ 400-700$\times$ slower than TCP due to the pure-Python `aioquic` implementation.

## 5.2 Impact of Block Size

- **TCP and UDP**: throughput increases significantly with larger blocks. Going from 12 KB to 60 KB blocks improves speed by 30-80%, because fewer syscalls and fewer loop iterations reduce Python-side overhead.
- **UDP loss**: at 1 GB streaming, smaller blocks produce more datagrams, leading to higher receiver-side buffer overflow. Loss drops from 37.88% (12 KB) to 0% (60 KB).
- **QUIC**: block size has *no measurable impact* on throughput. The constant 1.17 MB/s ceiling is set by the TLS processing overhead, not by the application-level framing.

## 5.3 Data Loss Analysis

Table 7: Data loss rates for UDP streaming at 1 GB

| Block (B) | Messages Lost | Bytes Lost | Loss (%) |
|---:|---:|---:|---:|
| 12 000 | 33 897 | 406 757 824 | 37.88 |
| 24 000 | 7 552 | 181 248 000 | 16.88 |
| 36 000 | 3 019 | 108 684 000 | 10.12 |
| 48 000 | 216 | 10 368 000 | 0.97 |
| 60 000 | 0 | 0 | 0.00 |

UDP streaming exhibits significant packet loss at 1 GB with small blocks. The loss is caused by the receiver's socket buffer being overwhelmed when datagrams arrive faster than the application can read them. Larger blocks reduce the total number of datagrams, giving the receiver more time per message and reducing or eliminating loss.

Both TCP and QUIC have zero data loss across all tests, as they provide built-in reliability mechanisms.

## 5.4 Streaming vs. Stop-and-Wait

Table 8: UDP: streaming vs. stop-and-wait at 1 GB

| Block (B) | Stream (MB/s) | Stream Loss | S&W (MB/s) | S&W Loss |
|---:|---:|---:|---:|---:|
| 12 000 | 209.19 | 37.88% | 36.38 | 0% |
| 24 000 | 264.68 | 16.88% | 73.97 | 0% |
| 36 000 | 373.59 | 10.12% | 120.79 | 0% |
| 48 000 | 82.18 | 0.97% | 153.48 | 0% |
| 60 000 | 479.20 | 0% | 209.37 | 0% |

The stop-and-wait mechanism provides a clear reliability guarantee: zero data loss at the cost of 2-6× lower throughput. For UDP, this trade-off is meaningful because streaming offers no delivery guarantees. For TCP, stop-and-wait is redundant since TCP already handles acknowledgments internally. For QUIC, stop-and-wait adds per-message TLS overhead on top of already slow processing, making it impractical.

# 6 Conclusions

## 6.1 Advantages and Disadvantages

Table 9: Protocol comparison summary

| Protocol | Advantages | Disadvantages |
|---|---|---|
| **TCP** | • Highest throughput (up to 801 MB/s)<br>• Zero data loss guaranteed<br>• Built-in flow/congestion control<br>• No need for stop-and-wait | • Connection setup overhead (3-way handshake)<br>• No built-in encryption |
| **UDP** | • Very fast streaming (up to 799 MB/s)<br>• Minimal protocol overhead<br>• No connection setup required | • No delivery guarantee<br>• High data loss under load (up to 37.88%)<br>• Requires application-level reliability (stop-and-wait) for correctness |
| **QUIC** | • Zero data loss (reliable like TCP)<br>• Built-in TLS 1.3 encryption<br>• Stream multiplexing<br>• No head-of-line blocking<br>• 0-RTT connection resumption | • Pure-Python `aioquic` is extremely slow ($\approx 1.17$ MB/s)<br>• TLS overhead per packet<br>• Block size has no impact on throughput<br>• Stop-and-wait is impractical ($\approx 100$ KB/s) |